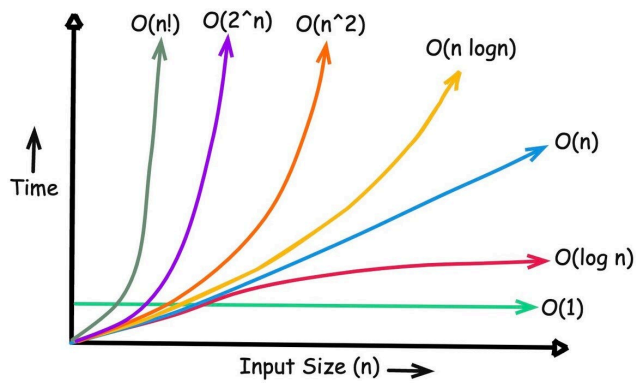




O	Ω	Θ
Big Oh	Omega	Theta
Upper Bound	Lower Bound	Upper & Lower Bound
Worst Case	Best Case	Average 'ish'

1. Circle/mark all the statements below that are true:

- a. $\log(n) = O(n)$ $\log n < n$
- b. $n = O(\log(n))$ $n < \log n$
- c. $\log^2(n) = O(n)$ $2 \log n < n$
- d. $n = O(\log^2(n))$ $n < 2 \log n$
- e. $\log(n) = \Omega(n)$ $\log n > n$
- f. $n = \Omega(\log(n))$ $n > \log n$
- g. $5n^3 + 7n + 13 = O(n^5)$ $5n^3 < n^5$
- h. $5n^3 + 7n + 13 = \Omega(n^5)$ $n^3 > n^5$
- i. $5n^3 + 7n + 13 = \Omega(n)$ $n^3 > n$
- j. $\log(n!) = \Theta(n \log(n))$
- k. $n^{1/2} = O(\log(n))$ $n^{1/2} < \log n$
- l. $2^n = \Omega(n!)$ $2^n > n!$

$\log(n!) = \Theta(n \log n)$ Both upper & lower bounds are the same
 $\log(n!) \leq O(n \log n)$
 $\geq \Omega(n \log n)$ true

2. Answer the following

a. What is the worst-case big-oh run time of Merge sort on n items?

$$O(n \log n)$$

b. What is the worst-case big-oh run time of Selection sort on n items?

$$O(n^2)$$

Linked List

For these questions, the data type **node** refers to the type defined by the class:

```
class node
{
public:
    int data;
    node * next;
};
```

A node contains two fields: **data** and **next**.

Consider the linked list below, with node pointers **ptrA** and **ptrB** pointing to the specified nodes in the list, and **head** pointing to the first node in the list.

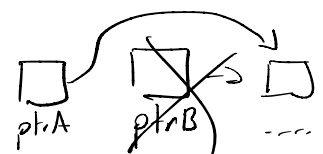
```
head
|
v
[15] -> [8] -> [32] -> [12] -> [2] -> [57] -> [23] -> [19] -> Null
                        ^       ^
                        ptrA   ptrB
```

a.

Assume **ptrB** points to the node with **2** in the data field and **ptrA** points to the node previous to that node. What C++ statements would delete the node with **2** from the list? Be sure your code does not create a memory leak.

$ptrA \rightarrow next = ptrB \rightarrow next;$

$delete\ ptrB;$



b.

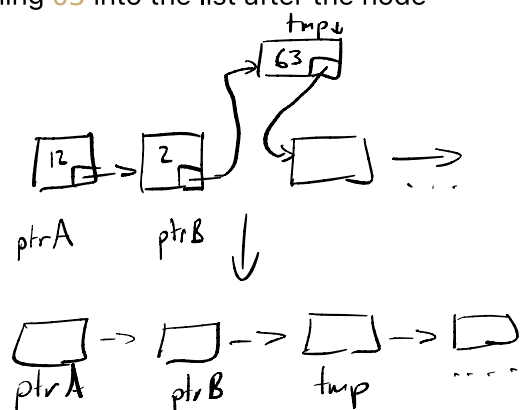
Assume **ptrB** points to the node with **2** in the data field and **ptrA** points to the node previous to that node. What C++ statements would create and insert a node containing **63** into the list after the node containing **2**?

$node* tmp = new\ node();$

$tmp \rightarrow data = 63;$

$tmp \rightarrow next = ptrB \rightarrow next;$

$ptrB \rightarrow next = tmp;$



// Analyze the big-O runtime for each function with respect to n:

```
void A(int n) O(1)
```

```
{
  if (n < 10) O(1)
```

```
    step++;
```

```
  else
```

```
  {
```

```
    for (int i = 0; i < n; i++) O(n)
```

```
      step++;
```

```
    A(n - 2); T(n-2)
```

```
  }
```

```
}
```

$$O(1) + O(n) + \dots + (n-2) = O(n^2)$$

$\downarrow$
 $O(n^2)$

So, the recursive equation is:

$$T(n) = T(n-2) + cn$$

$$T(n) = T(n-2) + cn$$

$$T(n-2) = T(n-4) + c(n-2)$$

$$T(n-4) = T(n-6) + c(n-4)$$

$$T(n-2\lambda) = 1$$

$$cn$$

$$c(n-2)$$

$$c(n-4)$$

⋮

$$1$$

Base case

$$i = 0$$

$$i = 1$$

$$i = 2$$

work done
per level

$$c(n-2(0))$$

$$c(n-2(1))$$

$$c(n-2(2))$$

↓

$$c(n-2\lambda)$$

of levels?

$$n-2\lambda = 1$$

$$2\lambda = 1+n$$

$$\lambda = 1 + \frac{n}{2}$$

work per lvl

$\times$ len. of tree

$$n$$

$$\times$$

$$\frac{n}{2}$$

$$= \frac{n^2}{2} \rightarrow O(n^2)$$

$cn \rightarrow$

```
#include <iostream>
#include <utility>

// Sorting
//
// For this question, suppose you have a linkedList class that includes methods
// 'split' and 'merge', already implemented (exactly as was done in the homework).
//
// split: Splits the contents of a list evenly into two given (initially empty)
// lists. If the number of items in the list is odd, the extra item goes in the
// first list. It runs in  $O(n)$  time, where  $n$  is the number of items in the list.
//
// merge: Takes two sorted lists and merges them into the calling (initially
// empty) list, creating one sorted list. It runs in  $O(n)$  time, where  $n$  is the
// total number of items in the two given lists.
//
// Add a sort() method to linkedList that sorts the items into ascending order
// with a fast running time. The split and merge methods may be used to help.
```

```
class linkedList {
private:
    class node {
    public:
        double data;
        node* next;
        node* prev;

        // Make a new node and set its data and links.
        node(double value) {
            data = value;
            next = nullptr;
            prev = nullptr;
        }
    };

    node* head; // Pointer to the first node in the list.
    node* tail; // Pointer to the last node in the list.
    int count;

public:
    // Moves this list's nodes alternately into A and B. When the size is odd,
    // A receives the extra node.
    void split(linkedList& A, linkedList& B) {...}

    // Merges sorted A and B into this initially empty list, moving their nodes.
    void merge(linkedList& A, linkedList& B) {...}

    // Merge sort:  $O(n \log n)$  time and  $O(\log n)$  call-stack space.
    void sort() {
        if (!head || !head->next) {  $O(1)$ 
            return;
        }

        linkedList A;
        linkedList B;

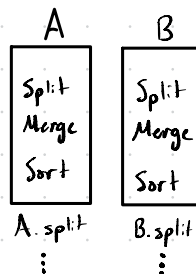
        split(A, B);

        A.sort();
        B.sort();

        merge(A, B);
    }
};
```

Steps:

- 1) Create subarrays
- 2) Split current list into A & B
- 3) Recursively sort subarrays
- 4) Merge subarrays



$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

$$\rightarrow O(n \log n)$$